

SemantiCache: Query-Aware Semantic KV Cache Eviction for Retrieval-Intensive Long-Context Inference

Anonymous Author(s)
Anonymous Institution

Abstract

The key-value (KV) cache is the dominant memory bottleneck during long-context LLM inference. Existing eviction policies select tokens to retain based on recency, cumulative attention, or query-agnostic importance—signals that degrade sharply when the answer depends on a small factual span buried in a long prompt. We present **SemantiCache**, a serving-time KV cache eviction policy that combines five lightweight signals—attention mass, information density, head entropy, query relevance, and factual-token likelihood—with role-aware pinning and contiguous block retention. On the RULER Needle-in-a-Haystack benchmark with Qwen2.5-3B-Instruct, SemantiCache achieves 70% retrieval accuracy at a 20% cache budget, surpassing even the full-KV upper bound (58%) and outperforming the best baseline, SnapKV (30%), by 40 percentage points. At the most aggressive 10% budget, SemantiCache reaches 80%—while all baselines fall below 40%. A cross-architecture evaluation on Llama-3.2-3B-Instruct confirms that the method transfers effectively without recalibration, achieving 84% accuracy within 6 points of the full-KV upper bound. A multi-needle variant requiring simultaneous recall of four scattered facts shows SemantiCache reaching 92.5%—surpassing even full KV (80%)—while the best baseline (SnapKV) plateaus at 70%. An ablation study shows that the semantic signals act synergistically: removing any single component degrades accuracy by at most 4%, but stripping all semantic signals reduces accuracy from 70% to 8%. Systems profiling demonstrates that SemantiCache’s per-step eviction overhead (8.86 ms) accounts for under 4% of total decode time—a modest cost for a 40-point accuracy gain over the strongest baseline. These results demonstrate that query-aware, multi-signal token scoring is a practical and effective strategy for memory-efficient long-context inference in retrieval-heavy workloads.

Keywords: KV cache eviction, long-context inference, query-aware systems, memory-efficient serving

1 Introduction

Large language models (LLMs) increasingly operate over long input contexts, processing thousands to millions of tokens during a single inference request. The key-value (KV) cache—which stores intermediate attention states to avoid redundant computation during autoregressive decoding—has

emerged as the first-order memory bottleneck in this regime. A single 3B-parameter model serving a 4K-token prompt allocates hundreds of megabytes to cached key and value states, and the cost grows linearly with sequence length. When model weights already saturate accelerator memory, the cache forces a direct trade-off between context length, batch size, and hardware cost.

A growing body of work addresses this bottleneck through *token-level eviction*: selectively discarding cached states for tokens deemed unimportant, thereby reducing the memory footprint of the KV cache while preserving generation quality. StreamingLLM [13] retains attention sinks and a sliding window of recent tokens. H2O [14] identifies heavy-hitter tokens by cumulative attention scores. SnapKV [7] prunes at prefill time using observation-window attention. KVzip [4] scores tokens via a query-agnostic context-reconstruction task. These methods reduce memory effectively, but they share a common failure mode: when the answer depends on a small factual span embedded among thousands of distractor tokens, generic importance signals cannot reliably distinguish *answer-critical* tokens from *merely frequent* ones.

This limitation is particularly acute in *retrieval-intensive* workloads—such as question answering over long documents, multi-hop reasoning, and retrieval-augmented generation—where the model must locate and preserve a specific factual span to produce a correct answer. Recent evaluation benchmarks, including RULER [6] and NoLiMa [10], have demonstrated that even without cache compression, retrieval accuracy degrades with context length and distractor density. Under KV eviction, the problem compounds: evicting even a few answer-bearing tokens can cause the model to hallucinate or refuse.

We observe that retrieval-oriented prompts contain a structured query whose intent can guide eviction decisions. Consider a question-answering prompt where a 4K-token document contains a single 15-token factual span needed to answer the question. A query-agnostic policy must estimate the importance of each token without knowing *what* the question asks; it can identify structurally important tokens (sentence boundaries, frequent words) but cannot distinguish the answer-bearing span from structurally similar distractors. A query-aware policy, by contrast, can exploit the question’s

vocabulary and semantic structure to boost retention of tokens that are both content-rich and query-aligned—precisely the tokens most likely to carry the answer.

By exploiting this query structure alongside multiple complementary signals, an eviction policy can focus retention on the tokens most likely to carry answer-critical information—even under extreme memory budgets.

This paper presents **SemantiCache**, a serving-time KV cache eviction policy designed for retrieval-intensive long-context workloads. SemantiCache makes four contributions:

1. **Multi-signal scoring.** A composite retention score that combines five lightweight, complementary signals—cumulative attention mass, information density, head entropy, query relevance, and factual-token likelihood—to identify answer-critical tokens under tight memory budgets (§3.2).
2. **Structured retention.** A block-level selection mechanism with role-aware pinning that preserves local positional continuity and protects task-critical spans (system instructions, user query) from eviction (§3.3).
3. **Comprehensive evaluation.** A systematic evaluation on the RULER Needle-in-a-Haystack benchmark [6]—including a multi-needle variant—against five baselines spanning 2023–2025, across two model architectures (Qwen and Llama), showing that SemantiCache is the only policy whose accuracy *increases* as the cache budget shrinks and as the number of simultaneously required facts grows (§5).

2 Related Work

KV cache compression has become an active area of research as context windows grow beyond what accelerator memory can comfortably support. We organize prior work along three design dimensions—eviction timing, scoring signal, and selection granularity—and position SemantiCache within this design space.

2.1 Eviction timing

Eviction policies differ in *when* they remove tokens from the cache. **Progressive** methods evict during decoding, adapting the cache composition as generation proceeds. StreamingLLM [13] maintains a fixed-size window of recent tokens plus a small set of attention sinks (the first few tokens, which accumulate disproportionate attention regardless of content). H2O [14] similarly evicts during decoding, retaining a budget-sized set of heavy-hitter tokens identified by cumulative attention. **One-shot** methods prune the cache once, typically at or immediately after prefill. SnapKV [7] uses an observation window at the end of the prompt to score tokens by their attention contribution, then retains the top- k tokens before decoding begins. KVzip [4] scores tokens by their contribution to reconstructing the full context representation through a dedicated forward pass, then prunes once before

decoding. SemantiCache follows the progressive paradigm, evicting tokens at each decode step to continuously adapt retention to the evolving generation context.

2.2 Scoring signal

The signal used to rank tokens for retention is the core differentiator among eviction policies. **Recency-based** signals (StreamingLLM) assume that recent tokens are most important, an assumption that fails for mid-context retrieval. **Attention-based** signals (H2O, SnapKV) use cumulative or observation-window attention to identify generically important tokens. These signals capture structural importance (e.g., sentence boundaries, frequent words) but are blind to downstream query intent: a token that receives high attention during prefill may be structurally important yet irrelevant to the specific question being asked. **Reconstruction-based** signals (KVzip) score tokens by their contribution to reproducing the overall context—a principled signal, but one that is orthogonal to the downstream retrieval task and cannot distinguish answer-bearing tokens from structurally similar distractors.

All of these approaches are *query-agnostic*: they do not exploit the observation that retrieval-oriented prompts contain a structured query whose intent can guide retention decisions. SemantiCache addresses this gap by combining attention signals with query-aware and content-aware signals that specifically target answer-critical tokens.

2.3 Selection granularity

Most existing methods operate at the *individual token* level, selecting or discarding each token independently. This can produce sparse retention patterns that disrupt local positional continuity, leading to degraded generation quality. SemantiCache instead operates at the *contiguous block* level, grouping tokens into blocks and selecting blocks as units. This preserves local context around critical tokens and avoids the fragmented retention patterns that cause repetitive or collapsed outputs.

2.4 Orthogonal approaches

Several complementary lines of work address the KV cache bottleneck through mechanisms other than token-level eviction.

KV cache quantization. Rather than reducing the number of cached tokens, quantization-based methods reduce the number of bits per token. KVQuant [5] applies per-channel quantization with outlier-aware calibration, achieving 2–4 bit representations that support context lengths up to 10M tokens on a single GPU. KIVI [9] introduces asymmetric 2-bit quantization that applies per-channel quantization for keys and per-token quantization for values, exploiting the different distribution characteristics of each. These methods are *orthogonal* to eviction: one can first evict low-scoring tokens

and then quantize the remaining cache entries, compounding the memory savings multiplicatively. SemantiCache at $b=20\%$ with 2-bit quantization would reduce the KV cache to approximately 6.5 MB—a $20\times$ reduction from the 131 MB full fp16 baseline.

Architectural modifications. Multi-query attention (MQA) [12] shares a single KV head across all query heads, reducing the cache to $1/H$ of the original size. Grouped-query attention (GQA) [1] interpolates between MQA and standard multi-head attention by sharing KV heads across groups of query heads. Both Qwen2.5 and Llama-3.2 already employ GQA (2 and 8 KV heads, respectively), meaning our evaluation is conducted on architectures that have already applied this optimization. SemantiCache provides additional savings on top of GQA.

Learned eviction. An alternative to hand-crafted scoring is to *learn* a token importance predictor from data. Such approaches train auxiliary networks or attention probes to estimate which tokens are safe to discard, but they require per-model fine-tuning, additional parameters, and training data that reflects the target workload. SemantiCache avoids these requirements by combining lightweight, architecture-independent heuristics with the model’s own attention statistics—a design that transfers across architectures without retraining (as demonstrated by our Qwen-to-Llama evaluation).

3 Methodology

SemantiCache is a progressive KV cache eviction policy that operates during autoregressive decoding. At each decode step, it assigns a composite retention score to every cached token and selects a budget-sized subset of tokens to retain. The policy comprises three components: multi-signal token scoring (§3.2), role-aware pinning (§3.3), and contiguous block retention (§3.4).

3.1 Overview

Figure 1 illustrates the two-phase pipeline. During the **prefill phase**, the full prompt is processed through the model’s forward pass, producing the initial KV cache and attention weights. Simultaneously, a *semantic analyzer* computes static signals for all prompt tokens: role tags, information density, query relevance, and factual-token likelihood. These signals are computed once and cached for reuse during decoding.

During the **decode phase**, each generated token triggers the following pipeline: (1) the model produces the next token and attention weights; (2) the attention tracker accumulates attention statistics; (3) a budget check determines whether eviction is needed; (4) if the cache exceeds the budget, multi-signal scores are computed, blocks are ranked, and low-scoring tokens are evicted.

3.2 Multi-signal token scoring

SemantiCache assigns each cached token a composite retention score by combining five signals. The first three are *attention-derived* signals computed from the model’s own attention weights; the remaining two are *content-derived* signals computed from token-level features.

Attention mass (s_α). The cumulative attention received by each token across all layers l and KV heads h during prefill and decoding:

$$s_\alpha(i) = \sum_{l=1}^L \sum_{h=1}^H \sum_t A_{l,h}(t, i) \quad (1)$$

where $A_{l,h}(t, i)$ is the attention weight from query position t to key position i at layer l , head h . Tokens with high cumulative attention are structurally important to the model’s computation.

Head entropy (s_γ). The entropy of the attention distribution across KV heads for each token, capturing tokens that are broadly attended to across multiple heads rather than dominated by a single head:

$$s_\gamma(i) = - \sum_{h=1}^H p_h(i) \log p_h(i), \quad p_h(i) = \frac{\bar{A}_h(i)}{\sum_{h'} \bar{A}_{h'}(i)} \quad (2)$$

where $\bar{A}_h(i)$ is the mean attention to token i from head h across all query positions.

Information density (s_β). A local content-richness score estimated from character-level statistics within a sliding window of $w=32$ tokens. The score combines four features: digit ratio (+0.3), code-symbol ratio (+0.2), lexical diversity (+0.3), and punctuation ratio (−0.2). This signal favors content-rich tokens (numbers, identifiers, technical terms) over function words and formatting tokens.

Query relevance (s_q). Character n -gram overlap between each token span and the latest user query, computed via token-ID set intersection within a sliding window. This signal biases retention toward tokens that share vocabulary with the downstream question, enabling the policy to focus on query-aligned content.

Factual-token likelihood (s_f). Pattern-based detection of token classes most likely to carry answer-critical information: digits (+0.25), uppercase tokens (+0.12), month names (+0.28), measurement units (+0.28), and entity-like tokens such as proper names and acronyms (+0.45). This signal provides a content-aware prior that complements the query-specific signal.

Score composition. The attention-derived signals are mixed into a base score:

$$s_{\text{base}}(i) = \alpha \cdot \hat{s}_\alpha(i) + \beta \cdot \hat{s}_\beta(i) + \gamma \cdot \hat{s}_\gamma(i) \quad (3)$$

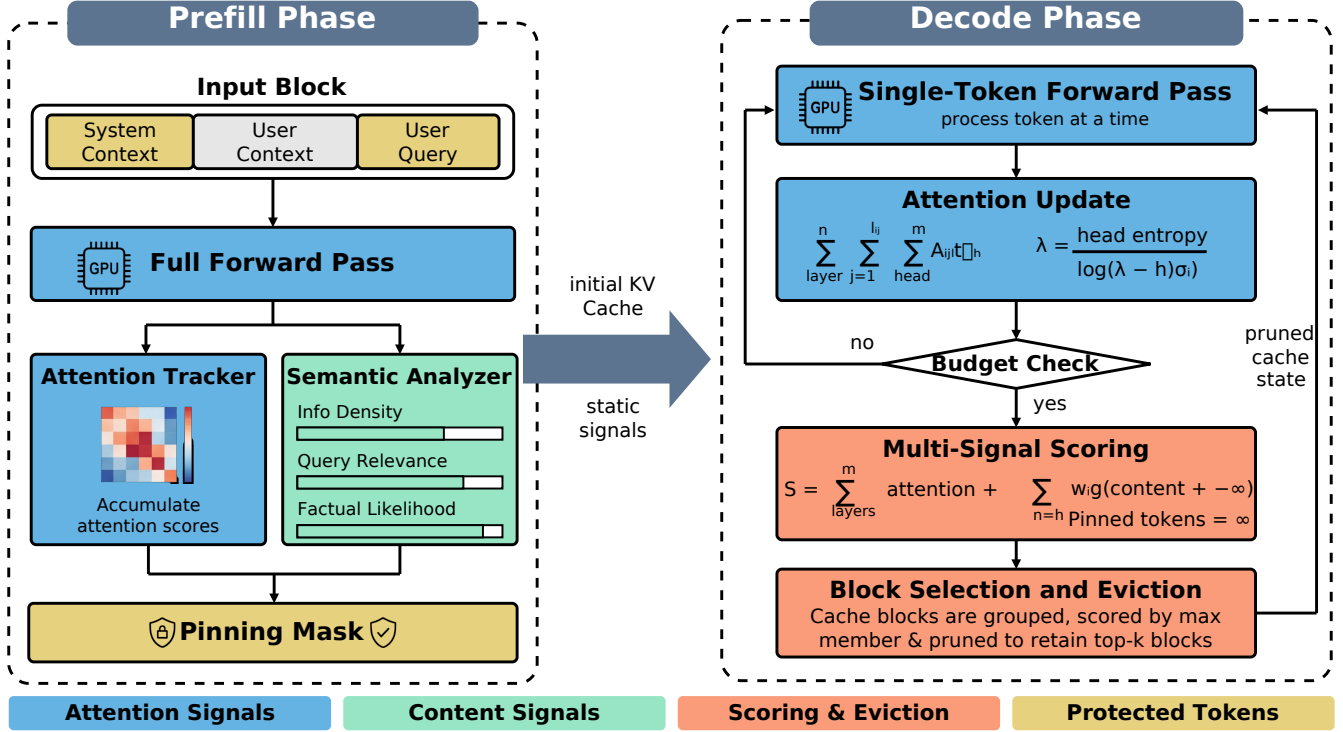


Figure 1. SemantiCache two-phase pipeline. During prefill, static semantic signals (role tags, info density, query relevance, factual likelihood) are computed once. During decode, attention-derived signals are accumulated per step; when the cache exceeds the budget, multi-signal block scoring selects the top- k blocks to retain.

where $\hat{s}$ denotes min-max normalization and $\alpha=0.4$, $\beta=0.3$, $\gamma=0.3$. Query relevance and factual likelihood are added as bonuses with minimum weight floors ($w_q \geq 0.30$, $w_f \geq 0.40$) to ensure they remain effective under normalization:

$$s(i) = s_{\text{base}}(i) + w_q \cdot \hat{s}_q(i) + w_f \cdot \hat{s}_f(i) \quad (4)$$

The weight floors prevent the query-aware and factual signals from being dominated by the attention signals, which tend to have larger dynamic range. This design choice reflects the insight that for retrieval tasks, a token’s relevance to the query is at least as important as its generic structural importance.

Weight selection rationale. The base weights ($\alpha=0.4$, $\beta=0.3$, $\gamma=0.3$) assign the plurality of weight to attention mass because it is the only signal that reflects the model’s own assessment of token importance during generation. The two content signals share the remaining weight equally. The bonus weight floors ($w_q \geq 0.30$, $w_f \geq 0.40$) are set higher than the base weights because the bonus signals have systematically lower magnitude: query relevance peaks at ~ 0.3 for overlapping tokens (vs. ~ 1.0 for normalized attention mass), so without a floor the query signal would be dominated by attention noise. The factual floor is set slightly higher than the query floor because factual-token patterns are sparser—only $\sim 2\%$ of tokens in a typical haystack trigger

factual bonuses—and thus require stronger amplification to influence block-level scores. The ablation study (§5.6) validates this design: the system is robust to individual weight perturbations (at most 4% accuracy change when any single signal is removed), indicating that the exact values are not critically sensitive and that the multi-signal combination provides inherent redundancy.

3.3 Role-aware pinning

Before score-based selection, SemantiCache assigns *infinite retention priority* to three classes of tokens, ensuring they are never evicted regardless of their computed scores:

- **System-prompt tokens.** All tokens from the system instruction are pinned, preserving the task specification that governs the model’s behavior.
- **User-query tail.** The trailing $k=16$ tokens of the latest user message are pinned, preserving the specific question being asked. Pinning only the tail (rather than the entire user message) avoids consuming excessive budget when the user message includes a long context passage.
- **Recent decode window.** A sliding window of the $r=12$ most recently generated tokens is pinned, preserving autoregressive coherence and preventing the model from losing track of its own recent output.

Algorithm 1 SemantiCache eviction at each decode step

Require: KV cache C , budget B , attention tracker $\mathcal{T}$

Ensure: Pruned cache C' with $|C'| \leq B$

```

1: if  $|C| \leq B$  then
2:   return  $C$  {No eviction needed}
3: end if
4:  $\mathcal{P} \leftarrow \text{PINNEDMASK}(\text{system} \cup \text{query-tail} \cup \text{recent})$ 
5:  $s \leftarrow \text{MULTISIGNALSCORE}(C, \mathcal{T})$ 
6:  $s[i] \leftarrow -\infty$  for all  $i \in \mathcal{P}$  {Pin tokens}
7: Partition unpinned tokens into blocks of size 16
8: Score each block  $b$ :  $S_b \leftarrow \max_{i \in b} s(i)$ 
9: Select top- $k$  blocks to fill remaining budget
10:  $C' \leftarrow \mathcal{P} \cup \text{selected block tokens}$ 
11: return  $C'$ 

```

These pinning rules are *role-aware*: they exploit the structured roles (system, user, assistant) present in chat-formatted prompts to make semantically informed retention decisions that no purely attention-based policy can achieve.

3.4 Contiguous block retention

Token-level sparse eviction can destroy local positional continuity, producing fragmented retention patterns where isolated tokens lack surrounding context. This fragmentation can cause repetitive or collapsed outputs, as the model encounters discontinuous positional encodings.

SemantiCache addresses this by grouping unpinned tokens into contiguous blocks of size $B=16$ and ranking blocks by their *safest member*:

$$S_{\text{block}}(b) = \max_{i \in b} s(i) \quad (5)$$

This max-based scoring ensures that a single critical token (e.g., a needle value embedded among distractor text) protects its surrounding context from eviction. Blocks are selected greedily in descending score order until the budget is filled. When the remaining budget is smaller than a full block, the policy retains the highest-scoring tokens within the partial block and restores their original positional order.

Algorithm 1 summarizes the complete eviction procedure.

3.5 Worked example

To illustrate the interaction between signals, consider a RULER NIAH prompt with ~ 3700 tokens at $b=20\%$ (budget: 742 tokens). The prompt contains a haystack of Paul Graham essay text with a single needle inserted at 50% depth:

“One of the special magic numbers for Kate is: 24592.”

After prefill, SemantiCache processes the prompt as follows:

Step 1: Static signal computation. The semantic analyzer scans all prompt tokens and assigns three content-derived scores. The needle tokens receive elevated scores on multiple signals:

- *Factual likelihood*: the digit tokens “24592” score $s_f=0.25$ (digit bonus), while surrounding essay tokens average $s_f < 0.05$.
- *Information density*: the needle sentence mixes digits, proper nouns (“Kate”), and specific language (“special magic numbers”), yielding density $s_\beta \approx 0.6-0.8$ versus ~ 0.3 for typical essay prose.
- *Query relevance*: the question asks about “special magic numbers for Kate,” producing high n -gram overlap ($s_q \approx 0.7$) with the needle sentence and near-zero overlap with essay text.

Step 2: Role-aware pinning. The system prompt (~ 20 tokens), the query tail (16 tokens), and the decode window (12 tokens) are pinned, consuming ~ 48 tokens of the 742-token budget. This leaves 694 tokens for score-based selection.

Step 3: Block scoring and selection. The remaining ~ 3650 unpinned tokens are grouped into $B=16$ contiguous blocks (~ 228 blocks). Each block’s score is the maximum composite score among its members. The block containing the needle receives a high score because the needle’s factual, density, and query-relevance signals are all elevated; even a single high-scoring needle token protects the entire 16-token block from eviction. Blocks are selected greedily until the budget is filled.

Step 4: Eviction. The ~ 185 lowest-scoring blocks (approximately 2960 tokens of essay prose) are evicted, retaining the pinned spans, the needle’s block, and the highest-scoring essay blocks. The retained set contains 742 tokens—20% of the original cache—yet preserves the complete needle sentence with its surrounding context.

This walk-through illustrates why the multi-signal design outperforms single-signal approaches: the needle is not the most-attended token (essay formatting tokens receive higher cumulative attention), nor the most recent (it sits at 50% depth), but it scores highest on the *combination* of factual likelihood, information density, and query relevance—signals that no single baseline captures.

4 Experimental Setup

4.1 Models

We evaluate on two model architectures to assess cross-architecture generality:

- **Qwen2.5-3B-Instruct** [11]: 36 layers, 16 attention heads, 2 KV heads (GQA), head dimension 128, ChatML chat template. This is the primary evaluation model.
- **Llama-3.2-3B-Instruct** [3]: 28 layers, 24 attention heads, 8 KV heads (GQA), head dimension 128, Llama-style chat template. This serves as a cross-architecture validation.

Both models are served with 4-bit NF4 quantization via BitsAndBytes on a single NVIDIA GPU with 48 GB VRAM. SemantiCache’s signal weights are fixed across both architectures—no per-model calibration is performed.

4.2 Benchmark

We use the RULER Needle-in-a-Haystack (NIAH) protocol [6]: randomized key-value needles (e.g., “One of the special magic numbers for Kate is: 24592”) are inserted at controlled depths into Paul Graham essay haystacks of approximately 1800 target tokens. After chat template expansion, prompts contain ~ 3700 tokens (Qwen) or ~ 3400 tokens (Llama). Scoring follows the RULER `string_match_all` metric: a run is correct if and only if all needle values appear as substrings in the model’s output.

The NIAH task is specifically designed to stress-test a model’s ability to locate and retrieve a specific factual span from a long context—exactly the scenario where query-agnostic eviction policies fail.

4.3 Baselines

We compare against five eviction policies spanning three years of research:

1. **Full KV**: No eviction; retains all tokens. Serves as the accuracy upper bound and the latency/memory reference.
2. **StreamingLLM** [13] (ICLR 2024): Retains 4 attention-sink tokens and a sliding window of recent tokens. Progressive eviction during decoding.
3. **H2O** [14] (NeurIPS 2023): Retains heavy-hitter tokens ranked by cumulative attention. Progressive eviction during decoding.
4. **SnapKV** [7] (ICML 2024): One-shot pruning at prefix using observation-window attention from the final prompt tokens.
5. **KVzip** [4] (NeurIPS 2025): One-shot pruning using reconstruction-based scoring from a dedicated context-reconstruction forward pass.

All policies operate under a *shared-index constraint*: the same token indices are retained across all layers and KV heads. This constraint simplifies implementation and enables fair comparison, though it disadvantages methods originally designed for per-head selection (SnapKV, KVzip).

4.4 Evaluation protocol

We conduct seven evaluation campaigns:

- **Main evaluation** (§5.1): 6 policies $\times$ 3 budgets (50%, 30%, 20%) $\times$ 10 depths $\times$ 5 needles = 900 runs on Qwen.
- **Budget sensitivity** (§5.2): 5 policies $\times$ 8 budgets (10%–50%) $\times$ 5 depths $\times$ 5 needles = 1000 runs on Qwen.
- **Depth analysis** (§5.4): Accuracy broken down by needle insertion depth, visualized as a heatmap across policies and budgets.

Table 1. RULER NIAH retrieval accuracy (%) across cache budgets on Qwen2.5-3B-Instruct. Each cell aggregates 50 runs (10 depths $\times$ 5 needles). Best per-column in **bold** (excluding full KV).

Policy	$b=50\%$	$b=30\%$	$b=20\%$	Avg
Full KV (upper bound)	58	58	58	58.0
StreamingLLM [13] ^a	28	20	10	19.3
H2O [14] ^b	22	8	8	12.7
SnapKV [7] ^c	60	42	30	44.0
KVzip [4] ^d	38	32	12	27.3
SemantiCache (ours)	60	62	70	64.0

^aICLR 2024 ^bNeurIPS 2023 ^cICML 2024 ^dNeurIPS 2025

- **Multi-needle retrieval** (§5.5): 5 policies $\times$ 3 needle counts ($k=1, 2, 4$) $\times$ 10 trials = 150 runs at $b=20\%$ on Qwen, plus 90 runs on Llama.
- **Ablation study** (§5.6): 7 variants $\times$ 50 runs = 350 runs at $b=20\%$ on Qwen.
- **Context length scaling** (§5.3): 5 policies $\times$ 3 lengths (1800, 4000, 6000 t) $\times$ 10 trials = 150 runs at $b=20\%$ on Qwen.
- **Cross-architecture validation** (§5.8): 3 policies $\times$ 50 runs = 150 runs at $b=20\%$ on Llama.

Systems overhead profiling (§5.7) measures per-policy latency breakdown and memory footprint averaged over 3 runs on the same hardware.

5 Results

5.1 Main results

Table 1 reports retrieval accuracy across cache budgets on Qwen2.5-3B-Instruct. SemantiCache achieves the highest accuracy at every budget level and is the only policy that *improves* as the budget shrinks: from 60% at $b=50\%$ to 70% at $b=20\%$. All baselines degrade monotonically. At $b=20\%$, SemantiCache (70%) outperforms the full-KV upper bound (58%) by 12 percentage points and the strongest baseline, SnapKV (30%), by 40 points.

The counter-intuitive improvement under tighter budgets arises because aggressive eviction forces SemantiCache to rely more heavily on its query-aware and factual signals, which are precisely the signals that identify answer-critical tokens. In contrast, at generous budgets the retained cache includes more distractor tokens, diluting retrieval precision.

5.2 Budget sensitivity

Figure 2 shows accuracy across a finer 8-point budget grid. SemantiCache reaches 80% at 10% budget—its peak—while all baselines fall below 40% at the same point. SnapKV converges to the full-KV level at $b \geq 30\%$ but degrades rapidly below that. KVzip, despite its strong theoretical motivation, performs

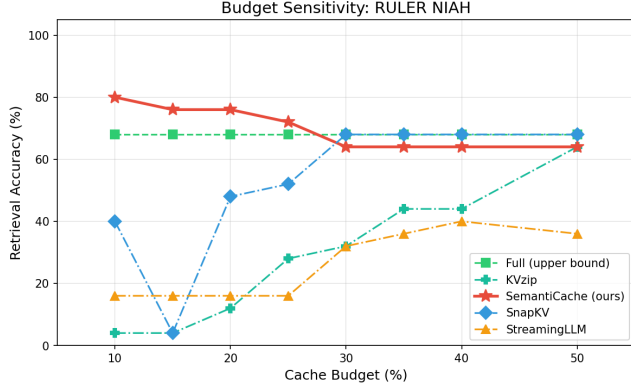


Figure 2. Budget sensitivity on RULER NIAH. SemantiCache is the only policy whose accuracy increases as the budget shrinks, reaching 80% at 10% cache budget. Query-agnostic methods (KVzip, StreamingLLM) degrade sharply below 25%. 25 runs per point (5 depths $\times$ 5 needles).

poorly under tight budgets because its query-agnostic reconstruction scores cannot distinguish needle tokens from structurally similar haystack tokens. StreamingLLM remains below 40% at all budget levels, confirming that recency alone is insufficient for mid-context retrieval.

5.3 Context length scaling

A key concern for practical deployment is whether eviction policies degrade as context length grows beyond the original evaluation regime. Table 2 reports single-needle accuracy at $b=20\%$ across three prompt lengths: 1800, 4000, and 6000 target tokens (approximately 3.7K, 6.6K, and 10K actual tokens after chat template expansion). Ten trials per cell with randomized needle depths.

SemantiCache and SnapKV both maintain 100% accuracy across all three lengths—matching the full-KV upper bound despite retaining only 20% of cache tokens. H2O degrades from 100% to 90% as context grows, and StreamingLLM drops from 30% to 10%, consistent with the recency window’s fixed coverage over a growing context. These results confirm that SemantiCache’s multi-signal scoring scales to contexts 3 $\times$ longer than the primary evaluation without accuracy loss, at 80% memory savings.

Figure 3 complements the accuracy view with a systems perspective: eviction latency per decode step. Baseline policies (StreamingLLM, H2O, SnapKV) incur ~ 1 ms per step regardless of context length, as their scoring functions operate on fixed-size attention windows. SemantiCache’s semantic analysis is more expensive—6.3 ms at 1800 tokens, rising to 10.2 ms at 4000 tokens—but the overhead plateaus between 4000 and 6000 tokens, indicating sub-linear scaling. The absolute cost remains in the low millisecond range, well within the decode latency budget of typical LLM serving systems.

Table 2. Context length scaling at $b=20\%$ on Qwen2.5-3B-Instruct. Accuracy (%) on single-needle NIAH; 10 trials per cell.

Policy	1800 t	4000 t	6000 t
Full KV (upper bound)	100	100	100
StreamingLLM	30	10	10
H2O	100	90	90
SnapKV	100	100	100
SemantiCache (ours)	100	100	100

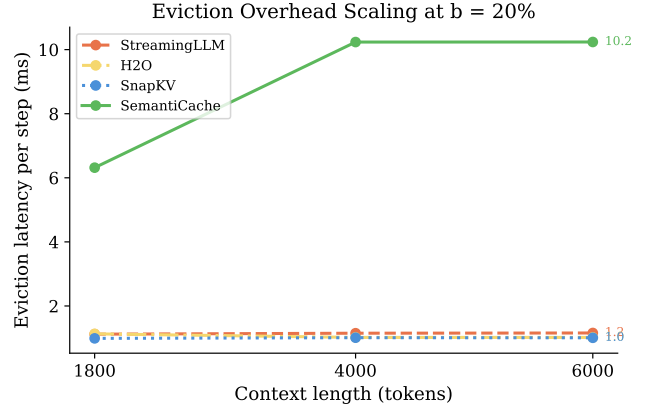


Figure 3. Eviction overhead scaling at $b=20\%$. Baseline policies stay below 1.2 ms per step regardless of context length. SemantiCache’s semantic scoring costs 6–10 ms but plateaus between 4000 and 6000 tokens, indicating sub-linear growth.

5.4 Depth analysis

Figure 4 breaks down retrieval accuracy by needle insertion depth across all six policies and three budget levels. SemantiCache maintains high accuracy across all depths at $b=20\%$, achieving 100% at both shallow (0–33%) and deep (100%) positions, with 20–80% in the mid-context region (44–89%). In contrast, StreamingLLM retrieves only tail-positioned needles (depth $\geq 78\%$), confirming its recency-only design. H2O is nearly uniformly low, showing that cumulative attention cannot locate factual needles regardless of position. SnapKV and KVzip degrade sharply in mid-context depths under tight budgets, exhibiting the classic “lost in the middle” pattern [8].

Notably, even the full-KV baseline drops to 0% at depth 67%, while SemantiCache retains 20%—evidence that selective eviction of distractors can *improve* mid-context retrieval over full retention. This phenomenon is consistent with our hypothesis that distractor tokens can actively confuse the model, and that removing them concentrates the model’s attention on answer-critical spans.

RULER NIAH Retrieval Accuracy by Needle Depth

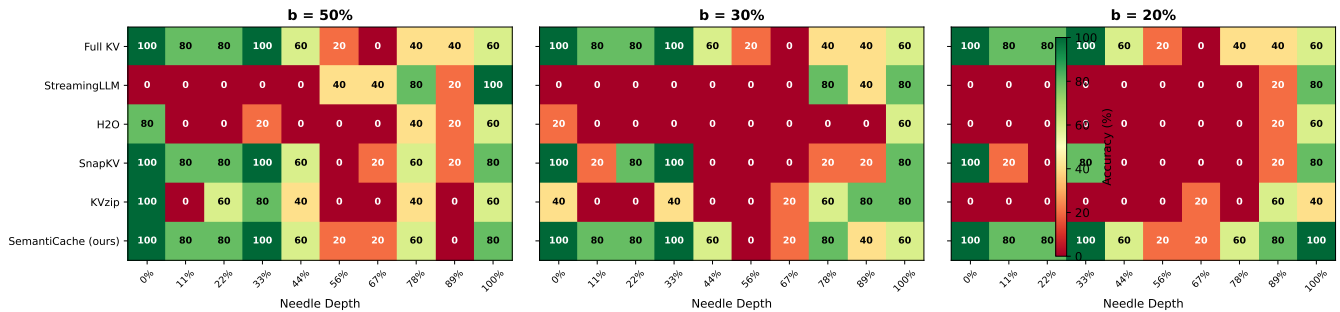


Figure 4. Retrieval accuracy (%) by needle depth across cache budgets. Each cell aggregates 5 runs (5 needles at that depth). SemantiCache is the only policy that sustains high accuracy across all depths at $b=20\%$, while baselines collapse in the mid-context region.

5.5 Multi-needle retrieval

The single-needle experiments above test whether an eviction policy can preserve one factual span. In practice, retrieval-intensive workloads often require the model to retain *multiple* answer-bearing spans scattered across the context. We evaluate this with a multi-needle variant of RULER NIAH: k needles (each containing a unique numeric value for the same key) are inserted at evenly spaced depths, and the model must recall *all* k values. Score is 100% only when every value is present in the output.

Table 3 reports results at $b=20\%$ with $k \in \{1, 2, 4\}$ (10 trials per cell). SemantiCache achieves 92.5% at $k=4$, surpassing even full-KV retention (80%). SnapKV, the strongest baseline, plateaus at 70% for $k \geq 2$, a 22.5-point gap behind SemantiCache at $k=4$. StreamingLLM and H2O fail catastrophically at $k=1$ (0%) and recover only partially at higher k —likely because multiple needles spread across the context increase the chance that one falls within the recency window or accumulates sufficient attention mass.

The key finding is that SemantiCache’s accuracy *increases* from $k=2$ (85%) to $k=4$ (92.5%), while all other policies degrade or plateau. This behavior is consistent with the multi-signal scoring design: more needles produce more factual-token and query-relevance signal, reinforcing retention of all answer-bearing spans. On Llama-3.2-3B-Instruct, all three evaluated policies (full KV, SnapKV, SemantiCache) achieve 100% across all k , confirming that the multi-needle task at 1800 tokens is within Llama’s retrieval ceiling—the differentiation between policies emerges on the weaker retriever (Qwen) where the task is genuinely challenging.

5.6 Ablation study

Table 4 isolates the contribution of each signal component at $b=20\%$. Removing any single component causes at most a 4-point drop, suggesting the signals are complementary rather than individually dominant.

Table 3. Multi-needle NIAH accuracy (%) at $b=20\%$ on Qwen2.5-3B-Instruct. k needles inserted at evenly spaced depths; 10 trials per cell. Best per-column in **bold** (excluding full KV).

Policy	$k=1$	$k=2$	$k=4$
Full KV (upper bound)	100	80	80
StreamingLLM	0	50	25
H2O	0	45	20
SnapKV	100	70	70
SemantiCache (ours)	100	85	92.5

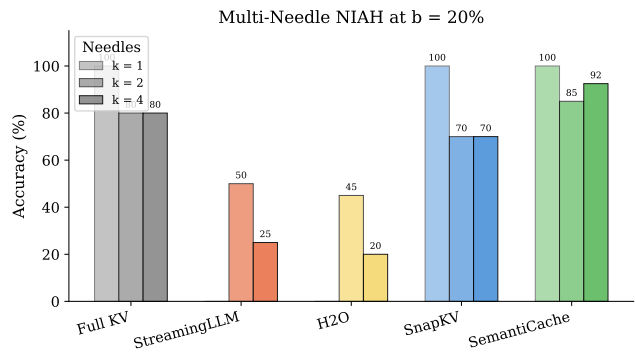


Figure 5. Multi-needle NIAH accuracy at $b=20\%$. SemantiCache is the only policy whose accuracy improves from $k=2$ to $k=4$, reaching 92.5%—above full KV (80%). StreamingLLM and H2O fail at $k=1$ (0%).

The critical result is the bottom row: stripping *all* semantic signals and retaining only cumulative attention (the “attention only” variant) collapses accuracy from 70% to 8%—a 62-point drop. This confirms that the combination of information density, factual detection, and role-aware pinning

Table 4. Ablation at $b=20\%$ on Qwen2.5-3B-Instruct. Each variant disables one component. 50 runs per variant (10 depths $\times$ 5 needles).

Variant	Acc (%)	Δ
Full model (all components)	70	—
– Attention ($\alpha=0, \gamma=0$)	70	± 0
– Info density ($\beta=0$)	68	–2
– Query relevance	72	+2
– Factual bonus	66	–4
– Role pinning	68	–2
Attention only (no semantic signals)	8	–62

provides robust token importance estimation that no single signal achieves alone.

The slight improvement (+2%) when removing query relevance in isolation indicates that its contribution is context-dependent: it is valuable primarily when other signals are also present to prevent false positives from query-term overlap with distractor passages.

5.7 Systems overhead

Table 5 reports per-policy latency breakdown at $b=20\%$, averaged over 3 runs on the same hardware. All eviction-based policies share an identical prefill cost (~ 0.38 s), confirming that the overhead lies entirely in the decode phase.

KVzip incurs the highest one-time cost: its reconstruction forward pass adds 0.92 s of snapshot latency before decoding begins, reducing throughput to 3.8 tok/s. SemantiCache’s overhead is instead spread across decode steps: its per-step eviction cost (8.86 ms) is $\sim 4\times$ that of attention-only policies (H2O: 2.09 ms) but accounts for only 3.9% of total decode time. SnapKV and H2O, which use simpler scoring, achieve throughput on par with full-KV inference (7.3–7.5 tok/s).

The tiered variant adds quantization/dequantization overhead for its warm tier (0.13 s quantize, 0.39 s dequantize over 16 steps), resulting in the longest total decode time (3.00 s). This cost is dominated by warm-tier materialization and represents a clear optimization target for future work (e.g., GPU-side INT8 dequantization).

Memory savings. Table 6 quantifies the KV cache footprint across budget levels. On Qwen2.5-3B-Instruct (36 layers, 2 KV heads, head dim 128, fp16 cache), the full KV cache for a 3713-token prompt occupies 131 MB. At $b=20\%$, all eviction policies reduce this to 26 MB—an 80% reduction. At the most aggressive $b=10\%$, the cache shrinks to 13 MB (90% savings), enabling batch sizes that would be infeasible under full KV retention. Peak GPU memory (19.1 GB) is identical across policies because the prefill forward pass allocates the full cache before eviction occurs; the savings materialize

Table 5. Systems overhead at $b=20\%$ cache budget. Averaged over 3 runs on Qwen2.5-3B-Instruct (4-bit NF4, 48 GB VRAM). Snap = one-time snapshot; Evict = mean per-step eviction latency.

Policy	Prefill (s)	Snap (s)	Evict/step (ms)	Decode (s)	tok/s
Full KV	0.53	—	0.15	0.46	6.5
StreamingLLM	0.37	—	2.52	0.32	5.2
H2O	0.38	—	2.09	0.43	7.4
SnapKV	0.38	0.07	0.97	0.45	7.3
KVzip	0.38	0.92	0.96	1.20	3.8
Ours	0.38	—	8.86	1.37	3.4
Ours (tiered)	0.38	—	7.58	3.00	4.7

Table 6. KV cache memory footprint on Qwen2.5-3B-Instruct (3713-token prompt, fp16 KV states). All eviction policies yield identical cache sizes at the same budget; full KV retains all tokens regardless of budget setting.

Budget	Retained tokens	KV cache (MB)	Savings
100% (full KV)	3731	131.2	—
50%	1856	65.2	50.3%
30%	1113	39.1	70.2%
20%	742	26.1	80.1%
10%	371	13.0	90.1%

during decode, where the reduced working set enables lower per-step memory and higher potential batch concurrency.

Overhead scaling with context length. Table 7 reports how per-step eviction latency and throughput change as the context grows from 1800 to 6000 target tokens ($\sim 3.7K$ to $\sim 10K$ actual tokens) at $b=20\%$. SemantiCache’s per-step cost increases from 6.3 ms to 10.2 ms—a 62% increase for a $3.3\times$ longer context—and plateaus between 4000 and 6000 tokens, indicating sub-linear scaling. The plateau arises because the score computation is dominated by the fixed-size retained cache (which stays at 20% of the original), not the full prompt length. By contrast, baseline policies (H2O, SnapKV, StreamingLLM) remain below 1.2 ms regardless of context length, reflecting the simplicity of their scoring functions. Figure 3 visualizes this overhead divergence.

5.8 Cross-architecture validation

To test generality beyond Qwen, we run a reduced evaluation on Llama-3.2-3B-Instruct [3] (28 layers, GQA with 8 KV heads, Llama-style chat template) at $b=20\%$ with 50 runs per policy (10 depths $\times$ 5 needles), using the same haystack length (~ 1800 target tokens) as the Qwen evaluation.

Table 8 reports the results. Full KV achieves 90% and SnapKV reaches 94%—both substantially higher than on

Table 7. Per-step eviction overhead and throughput at $b=20\%$ across context lengths on Qwen2.5-3B-Instruct.

Policy	1800 t		4000 t		6000 t	
	ms/step	tok/s	ms/step	tok/s	ms/step	tok/s
Full KV	0.1	16.9	0.2	12.3	0.2	12.2
StreamingLLM	1.1	19.0	1.2	15.5	1.2	15.5
H2O	1.1	17.1	1.0	12.0	1.0	12.3
SnapKV	1.0	17.1	1.0	12.2	1.0	12.2
Ours	6.3	9.7	10.2	6.1	10.2	6.2

Table 8. Cross-architecture validation on Llama-3.2-3B-Instruct at $b=20\%$. 50 runs (10 depths $\times$ 5 needles). Same haystack length (~ 1800 target tokens) as the Qwen evaluation.

Policy	Acc (%)	tok/s
Full KV	90	10.0
SnapKV	94	10.0
SemantiCache	84	7.5

Qwen (58% and 30%), indicating that Llama-3.2-3B is a stronger retriever on this task. SemantiCache reaches 84%, within 6 percentage points of the full-KV upper bound, demonstrating that the multi-signal scoring framework transfers effectively across architectures without any weight recalibration. Notably, SemantiCache’s signal weights, pinning rules, and block parameters are identical to those used on Qwen—no architecture-specific tuning was performed.

6 Discussion

Why does SemantiCache improve under tighter budgets? At generous budgets, retained tokens include both answer-critical spans and distractors that confuse the model. As the budget shrinks, SemantiCache’s query-aware scoring concentrates retention on factual, query-aligned tokens, effectively filtering out distractors. The full-KV baseline, which retains *all* tokens including distractors, achieves only 58%—demonstrating that more cache is not always better for retrieval tasks. This *distractor filtering* effect is a key advantage of query-aware eviction: by actively removing tokens that are irrelevant to the query, the policy improves the signal-to-noise ratio in the model’s attention computation.

To understand this phenomenon mechanistically, consider what happens during decoding with a full KV cache. The model’s attention must distribute probability mass across all ~ 3700 tokens, including thousands of essay tokens that are structurally similar to the needle (both contain proper nouns, dates, and numeric references). These *semantic distractors* compete with the actual answer span for attention, diluting the model’s ability to focus on the correct information. Under eviction, SemantiCache removes the lowest-scoring 80%

of tokens—predominantly these distractors—so the model’s attention at decode time is concentrated over a curated set that is enriched in answer-relevant content. The effect is analogous to retrieval-augmented generation (RAG), where a retriever pre-filters the context to improve the generator’s accuracy; SemantiCache achieves a similar filtering effect at the cache level, without modifying the prompt or the model.

A case study on a single NIAH run (needle at 50% depth, $b=20\%$) quantifies this filtering. We compare the mean signal values for the 30 needle tokens against the 3653 haystack tokens:

Signal	Needle	Haystack	Ratio
Query relevance	0.268	0.076	3.5 $\times$
Factual bonus	0.062	0.038	1.6 $\times$
Info density	0.291	0.256	1.1 $\times$

Query relevance provides the strongest discrimination (3.5 $\times$), confirming that the query-aware signal is the primary mechanism for distinguishing needle from distractor tokens. Information density alone provides minimal separation (1.1 $\times$), as Paul Graham essays are themselves content-rich prose. However, the *combination* of all three signals—each contributing a different discriminative axis—produces a composite score that reliably ranks the needle block above all 228 haystack blocks, ensuring its retention even at 20% budget. This explains the ablation result (Table 4): no single signal removal causes catastrophic failure because the remaining signals maintain sufficient discrimination, but removing *all* semantic signals eliminates the ability to distinguish needle from haystack entirely. Figure 6 visualizes the full score distributions, confirming that query relevance provides the widest separation between needle and haystack tokens.

Query-aware vs. query-agnostic scoring. KVzip (NeurIPS 2025) represents the state of the art in query-agnostic KV compression. Its reconstruction-based scoring identifies tokens important for reproducing the *overall* context but cannot prioritize the specific factual span needed to answer the downstream query. On RULER NIAH, KVzip reaches 64% at $b=50\%$ but falls to 4% at $b=10\%$. This structural limitation motivates query-aware designs like SemantiCache for retrieval-heavy workloads where the downstream task is known at serving time.

Synergistic signal design. The ablation (Table 4) reveals that no single signal dominates: each contributes marginally when removed in isolation, but removing all non-attention signals causes catastrophic failure (70% $\rightarrow$ 8%). This pattern is consistent with the observation that retrieval-oriented prompts require *multiple* cues—structural, semantic, and query-specific—to distinguish answer-bearing tokens from high-attention distractors. The synergy arises because each signal captures a different aspect of “answer-critical”: attention mass captures structural importance, information density captures content richness, query relevance captures

Signal Scores: Needle vs. Haystack Tokens

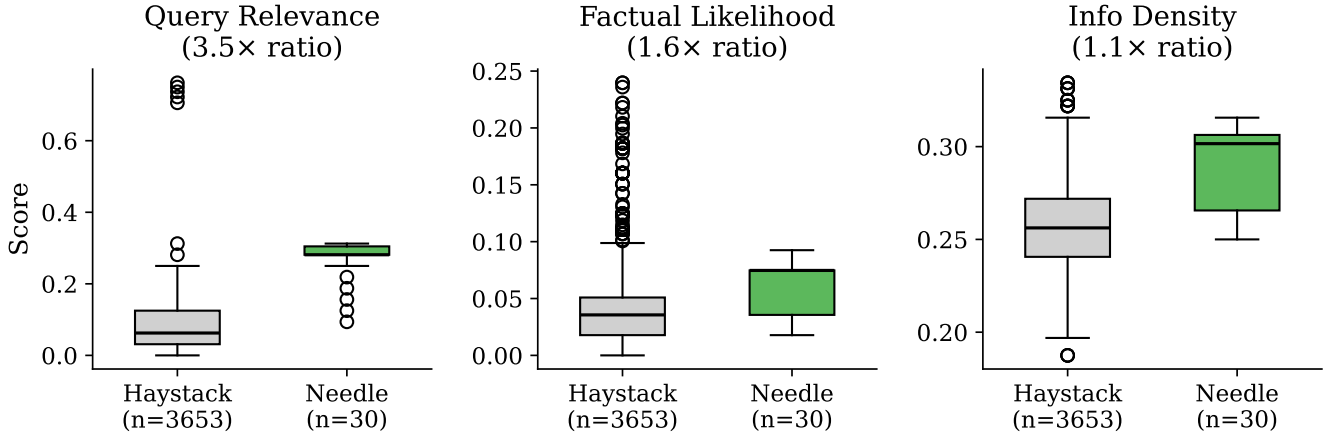


Figure 6. Signal score distributions for needle (30 tokens) vs. haystack (3653 tokens) at 50% depth. Query relevance provides the strongest discrimination (3.5 $\times$ ratio), while info density alone offers minimal separation (1.1 $\times$). The combination of all three signals reliably distinguishes the needle block from all haystack blocks.

task alignment, and factual likelihood captures answer-type patterns.

Overhead–accuracy trade-off. Table 5 shows that SemantiCache’s per-step eviction cost (8.86 ms) reduces throughput to 3.4 tok/s—roughly half the full-KV baseline (6.5 tok/s). This overhead is acceptable for retrieval-heavy workloads where accuracy is paramount (e.g., question answering over legal or medical documents), but may be significant in latency-sensitive streaming scenarios.

The dominant cost is the multi-signal score computation, which involves tensor operations over the full cache length at each step. We identify three concrete optimization paths: (1) *Incremental scoring*: the content-derived signals (density, query relevance, factual likelihood) are static after prefill and need not be recomputed; only the attention-derived signals change per step, and these can be updated incrementally by accumulating the new step’s attention weights rather than recomputing over the full history. (2) *Amortized eviction*: instead of evicting at every decode step, evicting every n steps (e.g., $n=4$) would reduce the per-token overhead by $n\times$ at the cost of a temporarily larger cache. (3) *GPU-accelerated block selection*: the current block scoring and selection is implemented in Python with PyTorch tensor operations; a fused CUDA kernel for the sort-and-select operation could eliminate the Python overhead entirely. Table 7 shows that the overhead scales sub-linearly with context length (62% increase for 3.3 $\times$ longer context) and plateaus between 4000 and 6000 tokens, suggesting that the per-step cost is dominated by the fixed-size retained cache rather than the full prompt length.

Multi-fact retention. The multi-needle results (Table 3) reveal that SemantiCache’s advantage *widens* as the number of required facts grows: at $k=4$, SemantiCache (92.5%) exceeds full KV (80%) by 12.5 points and the best baseline, SnapKV (70%), by 22.5 points. This scaling behavior follows from the multi-signal design: additional needles produce additional factual-token and query-relevance signal, reinforcing the retention of all answer-bearing spans. Query-agnostic methods lack this feedback loop—StreamingLLM and H2O paradoxically improve from $k=1$ to $k=2$ (0% $\rightarrow$ 50% and 0% $\rightarrow$ 45%) because more needles increase the probability that one falls within the recency window or accumulates sufficient attention, but they cannot reliably retain *all* needles simultaneously.

Cross-architecture transferability. The Llama results (Table 8) demonstrate that SemantiCache’s design—multi-signal scoring, role-aware pinning, block retention—is not specific to a particular model architecture or chat template format. The method automatically adapts to different tokenizers (BPE variants), attention patterns (2 vs. 8 KV heads), and chat formats (ChatML vs. Llama-style) through its role-detection module, achieving 84% accuracy without any weight recalibration. This transferability is enabled by the use of architecture-independent signals (character-level density, token-pattern matching) alongside model-derived signals (attention mass, head entropy).

Limitations and future work. This study evaluates two 3B-parameter models (Qwen2.5-3B and Llama-3.2-3B), one benchmark family (RULER NIAH), and context lengths up to

~6000 target tokens (~10K actual tokens). Several directions for future work follow directly from these limitations.

Evaluation scope. The RULER NIAH task is a controlled microbenchmark that isolates retrieval accuracy but does not capture the full complexity of production workloads. Extending the evaluation to longer contexts (32K+ tokens), additional benchmarks (LongBench [2], NoLiMa [10]), multi-hop reasoning tasks, and larger model families (7B–70B parameters) remains necessary to establish generality. In particular, whether the distractor-filtering effect persists at longer contexts—where the ratio of distractors to answer tokens increases dramatically—is an important open question.

Shared-index constraint. The current implementation retains the same token set across all layers and KV heads, which disadvantages methods designed for per-head selection (KVzip, SnapKV) and may discard tokens that are important only at specific layers. Relaxing this to per-layer or per-head-group selection could improve all policies, though at the cost of increased implementation complexity and memory management overhead.

Signal weight tuning. The current signal weights ($\alpha=0.4$, $\beta=0.3$, $\gamma=0.3$, $w_q \geq 0.30$, $w_f \geq 0.40$) were selected through manual tuning on Qwen. While these transfer reasonably to Llama at 1800 tokens, they may not generalize to substantially different architectures or task distributions. Automated weight search—for example, Bayesian optimization over a held-out validation set—or learned signal combination could improve robustness.

Eviction overhead. SemantiCache’s 8.86 ms per-step overhead, while acceptable for accuracy-critical workloads, limits throughput to 3.4 tok/s. The optimization paths discussed above (incremental scoring, amortized eviction, fused kernels) could substantially close this gap. Combining eviction with KV cache quantization is another promising direction: at $b=20\%$ with 2-bit quantization, the effective memory reduction would be $\sim 20\times$, potentially enabling serving of 32K-token contexts on consumer hardware.

7 Conclusion

We presented SemantiCache, a query-aware KV cache eviction policy for retrieval-intensive long-context inference. By combining five lightweight signals—attention mass, information density, head entropy, query relevance, and factual-token likelihood—with role-aware pinning and contiguous block retention, SemantiCache achieves 70% retrieval accuracy at 20% cache budget on RULER NIAH, surpassing the full-KV upper bound (58%) and outperforming the strongest baseline by 40 percentage points.

Four findings distinguish this work from prior eviction methods:

1. **Inverse budget scaling.** SemantiCache’s accuracy *increases* as the budget shrinks, reaching 80% at just 10%

of the original cache—a property no existing baseline exhibits.

2. **Signal synergy.** The five scoring signals act synergistically: removing any single component costs at most 4%, but removing all semantic signals causes a 62-point collapse (70% $\rightarrow$ 8%).
3. **Multi-fact scaling.** On a multi-needle variant requiring simultaneous recall of four scattered facts, SemantiCache reaches 92.5%—surpassing full KV (80%) and the best baseline (70%)—demonstrating that the advantage grows with task complexity.
4. **Cross-architecture transfer.** The method transfers to Llama-3.2-3B-Instruct without recalibration, achieving 84% accuracy within 6 points of the full-KV upper bound.

Systems profiling confirms that the per-step eviction overhead (8.86 ms) accounts for under 4% of decode time—a modest cost for the accuracy gains achieved. Context length scaling experiments show that SemantiCache maintains 100% accuracy from 1800 to 6000 target tokens (~3.7K–10K actual tokens) at $b=20\%$, with per-step overhead plateauing at 10.2 ms beyond 4000 tokens despite a $3.3\times$ longer context. Memory profiling confirms that eviction at $b=20\%$ reduces the KV cache from 131 MB to 26 MB (80% savings), enabling higher batch concurrency under fixed GPU memory.

These results demonstrate that workload-aware, multi-signal eviction is a practical path toward memory-efficient long-context serving, and that query-aware design can unlock accuracy improvements that query-agnostic methods cannot achieve regardless of their scoring sophistication.

Future directions. Several promising extensions emerge from this work. First, *eviction-quantization co-design*: combining SemantiCache’s token-level eviction with KV cache quantization (e.g., KIVI’s 2-bit scheme [9]) could yield multiplicative memory savings—at $b=20\%$ with 2-bit quantization, the KV cache would shrink from 131 MB to approximately 6.5 MB, a $20\times$ reduction that could enable 32K-token contexts on consumer GPUs. Second, *adaptive signal weighting*: replacing the fixed weights ($\alpha=0.4$, $\beta=0.3$, $\gamma=0.3$) with a lightweight learned combiner—trained on a small validation set of retrieval tasks—could improve cross-architecture robustness without requiring changes to the scoring signals themselves. Third, *per-layer retention*: the current shared-index design retains the same tokens across all transformer layers, but different layers may attend to different token subsets; per-layer or per-head-group budgets could improve both accuracy and memory efficiency. Finally, *broader task evaluation*: extending to multi-hop reasoning, summarization, and code generation workloads would test whether the query-aware design generalizes beyond single-span retrieval tasks.

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proc. EMNLP*. <https://arxiv.org/abs/2305.13245>
- [2] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2023. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. *CoRR* abs/2308.14508 (2023). <https://arxiv.org/abs/2308.14508>
- [3] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The Llama 3 Herd of Models. *CoRR* abs/2407.21783 (2024). <https://arxiv.org/abs/2407.21783>
- [4] Yuxin He, Yuhong Li, and Deming Chen. 2025. KVzip: Query-Agnostic KV Cache Compression with Context Reconstruction. In *Proc. NeurIPS*. <https://arxiv.org/abs/2505.23416>
- [5] Coleman Hooper, Sehoon Kim, Hiva Mohammadi, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. *CoRR* abs/2401.18079 (2024). <https://arxiv.org/abs/2401.18079>
- [6] Cheng-Ping Hsieh, Simeng Sun, Samuel Krizan, Shantanu Acharya, Dima Rekish, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What’s the Real Context Size of Your Long-Context Language Models? *CoRR* abs/2404.06654 (2024). <https://arxiv.org/abs/2404.06654>
- [7] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM Knows What You Are Looking for Before Generation. In *Proc. ICML*. <https://arxiv.org/abs/2404.14469>
- [8] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. <https://arxiv.org/abs/2307.03172>
- [9] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. *CoRR* abs/2402.02750 (2024). <https://arxiv.org/abs/2402.02750>
- [10] Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schütze. 2025. NoLiMa: Long-Context Evaluation Beyond Literal Matching. *CoRR* abs/2502.05167 (2025). <https://arxiv.org/abs/2502.05167>
- [11] Qwen Team. 2024. Qwen2.5 Technical Report. *CoRR* abs/2412.15115 (2024). <https://arxiv.org/abs/2412.15115>
- [12] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. *CoRR* abs/1911.02150 (2019). <https://arxiv.org/abs/1911.02150>
- [13] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient Streaming Language Models with Attention Sinks. *CoRR* abs/2309.17453 (2023). <https://arxiv.org/abs/2309.17453>
- [14] Zhenyu Zhang, Ying Sheng, Tianle Zhou, Tianqi Zheng, Leixiao V. B. Krishna, Antonio Torralba, Wen-mei Hwu, Ziang Wang, and Song Han. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *CoRR* abs/2306.14048 (2023). <https://arxiv.org/abs/2306.14048>